

# Multidimensional Difference Arrays

via Discrete Calculus



(<https://github.com/mazmap/multidimensional-difference-arrays>)

↑ Slides ↑

# Who am I?

- Matteo Mertz

# Who am I?

- Matteo Mertz
- Finishing my bachelors degree in mathematics at TUM

## Who am I?

- Matteo Mertz
- Finishing my bachelors degree in mathematics at TUM
- Currently writing my bachelors thesis on the *Efficient computation of Euler characteristic profiles (of cubical filtrations of digital images)*

## What is this talk about?

- A runtime optimization for applying  $L$  update queries to an array of size  $N$ :

$$\mathcal{O}(L \cdot N) \longrightarrow \mathcal{O}(L + N)$$

## What is this talk about?

- A runtime optimization for applying  $L$  update queries to an array of size  $N$ :

$$\mathcal{O}(L \cdot N) \longrightarrow \mathcal{O}(L + N)$$

- Simple heuristic for 1D, 2D and 3D arrays

## What is this talk about?

- A runtime optimization for applying  $L$  update queries to an array of size  $N$ :

$$\mathcal{O}(L \cdot N) \longrightarrow \mathcal{O}(L + N)$$

- Simple heuristic for 1D, 2D and 3D arrays
- For general dimension: *Discrete Calculus*

## What is this talk about?

- A runtime optimization for applying  $L$  update queries to an array of size  $N$ :

$$\mathcal{O}(L \cdot N) \longrightarrow \mathcal{O}(L + N)$$

- Simple heuristic for 1D, 2D and 3D arrays
- For general dimension: *Discrete Calculus*
- Further optimizations

## One-dimensional difference arrays

|    |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|
|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| A: | * | * | * | * | * | * | * | * | * |



Apply query

|    |   |    |    |    |   |   |   |   |   |
|----|---|----|----|----|---|---|---|---|---|
|    | 0 | 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 |
| A: | * | +C | +C | +C | * | * | * | * | * |

## One-dimensional difference arrays

|    |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|
|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| A: | * | * | * | * | * | * | * | * | * |



Apply query

|    |   |    |    |    |   |   |   |   |   |
|----|---|----|----|----|---|---|---|---|---|
|    | 0 | 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 |
| A: | * | +C | +C | +C | * | * | * | * | * |

**General idea** for an array of size  $N$  and  $L$  queries:

“Mark”  $L$  queries in  $\mathcal{O}(L)$  and perform the updates later in one sweep in  $\mathcal{O}(N)$ .

## One-dimensional difference arrays

|    |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|
|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| A: | * | * | * | * | * | * | * | * | * |

↓ Apply query

|    |   |    |    |    |   |   |   |   |   |
|----|---|----|----|----|---|---|---|---|---|
|    | 0 | 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 |
| A: | * | +C | +C | +C | * | * | * | * | * |

|    |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|
|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| d: | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

**General idea** for an array of size  $N$  and  $L$  queries:

“Mark”  $L$  queries in  $\mathcal{O}(L)$  and perform the updates later in one sweep in  $\mathcal{O}(N)$ .

## One-dimensional difference arrays

|    |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|
|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| A: | * | * | * | * | * | * | * | * | * |

↓ Apply query

|    |   |    |    |    |   |   |   |   |   |
|----|---|----|----|----|---|---|---|---|---|
|    | 0 | 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 |
| A: | * | +C | +C | +C | * | * | * | * | * |

|    |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|
|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| d: | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

↓ Mark query

|    |   |    |   |   |    |   |   |   |   |
|----|---|----|---|---|----|---|---|---|---|
|    | 0 | 1  | 2 | 3 | 4  | 5 | 6 | 7 | 8 |
| d: | 0 | +C | 0 | 0 | -C | 0 | 0 | 0 | 0 |

**General idea** for an array of size  $N$  and  $L$  queries:

“Mark”  $L$  queries in  $\mathcal{O}(L)$  and perform the updates later in one sweep in  $\mathcal{O}(N)$ .

## One-dimensional difference arrays

|    |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|
|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| A: | * | * | * | * | * | * | * | * | * |

↓ Apply query

|    |   |    |    |    |   |   |   |   |   |
|----|---|----|----|----|---|---|---|---|---|
|    | 0 | 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 |
| A: | * | +C | +C | +C | * | * | * | * | * |

**General idea** for an array of size  $N$  and  $L$  queries:

“Mark”  $L$  queries in  $\mathcal{O}(L)$  and perform the updates later in one sweep in  $\mathcal{O}(N)$ .

|    |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|
|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| d: | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

↓ Mark query

|    |   |    |   |   |    |   |   |   |   |
|----|---|----|---|---|----|---|---|---|---|
|    | 0 | 1  | 2 | 3 | 4  | 5 | 6 | 7 | 8 |
| d: | 0 | +C | 0 | 0 | -C | 0 | 0 | 0 | 0 |

↓ Cumulative sum

|              |   |   |   |   |   |   |   |   |   |
|--------------|---|---|---|---|---|---|---|---|---|
|              | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| $\Sigma d$ : | 0 | C | C | C | 0 | 0 | 0 | 0 | 0 |

## One-dimensional difference arrays

|    |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|
|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| A: | * | * | * | * | * | * | * | * | * |

↓ Apply query

|    |   |    |    |    |   |   |   |   |   |
|----|---|----|----|----|---|---|---|---|---|
|    | 0 | 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 |
| A: | * | +C | +C | +C | * | * | * | * | * |

**General idea** for an array of size  $N$  and  $L$  queries:

“Mark”  $L$  queries in  $\mathcal{O}(L)$  and perform the updates later in one sweep in  $\mathcal{O}(N)$ .

|    |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|
|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| d: | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

↓ Mark query

|    |   |    |   |   |    |   |   |   |   |
|----|---|----|---|---|----|---|---|---|---|
|    | 0 | 1  | 2 | 3 | 4  | 5 | 6 | 7 | 8 |
| d: | 0 | +C | 0 | 0 | -C | 0 | 0 | 0 | 0 |

↓ Cumulative sum

|              |   |   |   |   |   |   |   |   |   |
|--------------|---|---|---|---|---|---|---|---|---|
|              | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| $\Sigma d$ : | 0 | C | C | C | 0 | 0 | 0 | 0 | 0 |

↓ Applying the total update

|                  |   |    |    |    |   |   |   |   |   |
|------------------|---|----|----|----|---|---|---|---|---|
|                  | 0 | 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 |
| A + $\Sigma d$ : | * | +C | +C | +C | * | * | * | * | * |

## One-dimensional difference arrays

For an array of size  $N$  and  $L$  queries of arbitrary size

### Naive algorithm:

1. For every query add its update value over the whole update range

Runtime:

## One-dimensional difference arrays

For an array of size  $N$  and  $L$  queries of arbitrary size

### Naive algorithm:

1. For every query add its update value over the whole update range

Runtime:  $\mathcal{O}(L \cdot N)$

## One-dimensional difference arrays

For an array of size  $N$  and  $L$  queries of arbitrary size

### Naive algorithm:

1. For every query add its update value over the whole update range

Runtime:  $\mathcal{O}(L \cdot N)$

### Non-formal difference array algorithm:

1. Initialize array  $d$  of size  $N$  with zero entries
2. For every query add two markers to  $d$
3. Form the cumulative sum  $\Sigma d$
4. Add  $\Sigma d$  to  $A$

Runtime:

## One-dimensional difference arrays

For an array of size  $N$  and  $L$  queries of arbitrary size

### Naive algorithm:

1. For every query add its update value over the whole update range

Runtime:  $\mathcal{O}(L \cdot N)$

### Non-formal difference array algorithm:

1. Initialize array  $d$  of size  $N$  with zero entries in  $\mathcal{O}(N)$
2. For every query add two markers to  $d$
3. Form the cumulative sum  $\Sigma d$
4. Add  $\Sigma d$  to  $A$

Runtime:

## One-dimensional difference arrays

For an array of size  $N$  and  $L$  queries of arbitrary size

### Naive algorithm:

1. For every query add its update value over the whole update range

Runtime:  $\mathcal{O}(L \cdot N)$

### Non-formal difference array algorithm:

1. Initialize array  $d$  of size  $N$  with zero entries in  $\mathcal{O}(N)$
2. For every query add two markers to  $d$  in  $\mathcal{O}(L)$
3. Form the cumulative sum  $\Sigma d$
4. Add  $\Sigma d$  to  $A$

Runtime:

## One-dimensional difference arrays

For an array of size  $N$  and  $L$  queries of arbitrary size

### Naive algorithm:

1. For every query add its update value over the whole update range

Runtime:  $\mathcal{O}(L \cdot N)$

### Non-formal difference array algorithm:

1. Initialize array  $d$  of size  $N$  with zero entries in  $\mathcal{O}(N)$
2. For every query add two markers to  $d$  in  $\mathcal{O}(L)$
3. Form the cumulative sum  $\Sigma d$  in  $\mathcal{O}(N)$
4. Add  $\Sigma d$  to  $A$

Runtime:

## One-dimensional difference arrays

For an array of size  $N$  and  $L$  queries of arbitrary size

### Naive algorithm:

1. For every query add its update value over the whole update range

Runtime:  $\mathcal{O}(L \cdot N)$

### Non-formal difference array algorithm:

1. Initialize array  $d$  of size  $N$  with zero entries in  $\mathcal{O}(N)$
2. For every query add two markers to  $d$  in  $\mathcal{O}(L)$
3. Form the cumulative sum  $\Sigma d$  in  $\mathcal{O}(N)$
4. Add  $\Sigma d$  to  $A$  in  $\mathcal{O}(N)$

Runtime:

## One-dimensional difference arrays

For an array of size  $N$  and  $L$  queries of arbitrary size

### Naive algorithm:

1. For every query add its update value over the whole update range

Runtime:  $\mathcal{O}(L \cdot N)$

### Non-formal difference array algorithm:

1. Initialize array  $d$  of size  $N$  with zero entries in  $\mathcal{O}(N)$
2. For every query add two markers to  $d$  in  $\mathcal{O}(L)$
3. Form the cumulative sum  $\Sigma d$  in  $\mathcal{O}(N)$
4. Add  $\Sigma d$  to  $A$  in  $\mathcal{O}(N)$

Runtime:  $\mathcal{O}(L + N)$

## One-dimensional difference arrays

- Imagine placing the markers as forming separate difference arrays  $d_1, \dots, d_L$ , that is

$$d = \sum_{i=1}^L d_i$$

## One-dimensional difference arrays

- Imagine placing the markers as forming separate difference arrays  $d_1, \dots, d_L$ , that is

$$d = \sum_{i=1}^L d_i$$

- The difference array algorithm works as the cumulative sum distributes over the markers of multiple queries:

$$\Sigma d[j] = \sum_{l=1}^j d[l] = \sum_{l=1}^j \sum_{i=1}^L d_i[l] = \sum_{i=1}^L \sum_{l=1}^j d_i[l] = \sum_{i=1}^L \Sigma d_i[j]$$

## One-dimensional difference arrays

- Imagine placing the markers as forming separate difference arrays  $d_1, \dots, d_L$ , that is

$$d = \sum_{i=1}^L d_i$$

- The difference array algorithm works as the cumulative sum distributes over the markers of multiple queries:

$$\Sigma d[j] = \sum_{l=1}^j d[l] = \sum_{l=1}^j \sum_{i=1}^L d_i[l] = \sum_{i=1}^L \sum_{l=1}^j d_i[l] = \sum_{i=1}^L \Sigma d_i[j]$$

- The cumulative sum  $\Sigma d$  can be computed in  $\mathcal{O}(N)$  by reusing previous results:

$$\Sigma d[j] = d[j] + \sum_{i=1}^{j-1} d[i] = d[j] + \Sigma d[j-1].$$

## Two-dimensional difference arrays

This concept can easily be generalized to 2D arrays.

|   |    |    |    |    |    |   |   |
|---|----|----|----|----|----|---|---|
| * | *  | *  | *  | *  | *  | * | * |
| * | +C | +C | +C | +C | +C | * | * |
| * | +C | +C | +C | +C | +C | * | * |
| * | +C | +C | +C | +C | +C | * | * |
| * | *  | *  | *  | *  | *  | * | * |

Naively applying the constant update to a  $5 \times 8$  array A.

## Two-dimensional difference arrays

This concept can easily be generalized to 2D arrays.

|   |      |      |      |      |      |   |   |
|---|------|------|------|------|------|---|---|
| * | *    | *    | *    | *    | *    | * | * |
| * | $+C$ | $+C$ | $+C$ | $+C$ | $+C$ | * | * |
| * | $+C$ | $+C$ | $+C$ | $+C$ | $+C$ | * | * |
| * | $+C$ | $+C$ | $+C$ | $+C$ | $+C$ | * | * |
| * | *    | *    | *    | *    | *    | * | * |

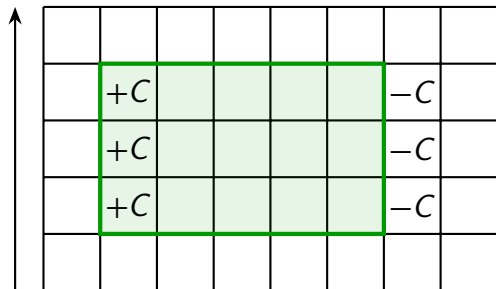
Naively applying the constant update to a  $5 \times 8$  array A.

|  |      |  |  |  |  |      |  |
|--|------|--|--|--|--|------|--|
|  | $-C$ |  |  |  |  | $+C$ |  |
|  |      |  |  |  |  |      |  |
|  |      |  |  |  |  |      |  |
|  |      |  |  |  |  |      |  |
|  |      |  |  |  |  |      |  |

Placing *four* markers in the difference array.

## Two-dimensional difference arrays

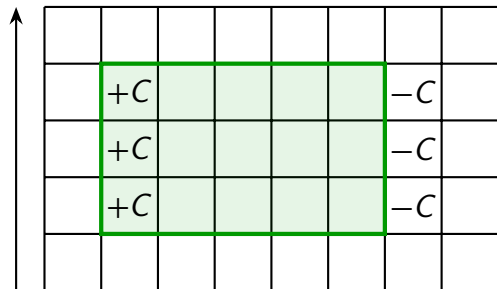
Taking cumulative sums in both directions yields the correct updates.



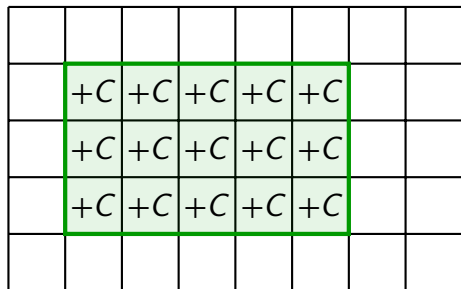
Cumulative sum along the  $y$ -axis

## Two-dimensional difference arrays

Taking cumulative sums in both directions yields the correct updates.



Cumulative sum along the y-axis



Cumulative sum along the x-axis

## **$d$ -dimensional difference arrays?**

- This heuristic can easily be extended to  $d$ -dimensional arrays by placing  $2^d$  markers for every query

## $d$ -dimensional difference arrays?

- This heuristic can easily be extended to  $d$ -dimensional arrays by placing  $2^d$  markers for every query
- However for dimensions  $d \geq 4$  this is hard to conceptually verify

## **$d$ -dimensional difference arrays?**

- This heuristic can easily be extended to  $d$ -dimensional arrays by placing  $2^d$  markers for every query
- However for dimensions  $d \geq 4$  this is hard to conceptually verify
- Furthermore, “naive” proofs without any additional tools are bound to get very messy

## $d$ -dimensional difference arrays?

- This heuristic can easily be extended to  $d$ -dimensional arrays by placing  $2^d$  markers for every query
- However for dimensions  $d \geq 4$  this is hard to conceptually verify
- Furthermore, “naive” proofs without any additional tools are bound to get very messy
- This is where *Discrete Calculus* comes into play

## $d$ -dimensional difference arrays?

- This heuristic can easily be extended to  $d$ -dimensional arrays by placing  $2^d$  markers for every query
- However for dimensions  $d \geq 4$  this is hard to conceptually verify
- Furthermore, “naive” proofs without any additional tools are bound to get very messy
- This is where *Discrete Calculus* comes into play
- To leverage these tools, we first need to formally describe our situation

# The formal framework

We view arrays as maps over *finite grid domains*.

## Definition (discrete maps, arrays):

- For  $a \leq b \in \mathbb{Z}$  define the *discrete interval*  $\llbracket a, b \rrbracket := \{m \in \mathbb{Z} \mid a \leq m \leq b\}$

# The formal framework

We view arrays as maps over *finite grid domains*.

## Definition (discrete maps, arrays):

- For  $a \leq b \in \mathbb{Z}$  define the *discrete interval*  $\llbracket a, b \rrbracket := \{m \in \mathbb{Z} \mid a \leq m \leq b\}$
- A *finite grid domain* is a subset  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$

# The formal framework

We view arrays as maps over *finite grid domains*.

## Definition (discrete maps, arrays):

- For  $a \leq b \in \mathbb{Z}$  define the *discrete interval*  $\llbracket a, b \rrbracket := \{m \in \mathbb{Z} \mid a \leq m \leq b\}$
- A *finite grid domain* is a subset  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$
- A *discrete map* is a map  $f : D \rightarrow \mathbb{R}$  over a finite grid domain  $D \subseteq \mathbb{Z}^d$

# The formal framework

We view arrays as maps over *finite grid domains*.

## Definition (discrete maps, arrays):

- For  $a \leq b \in \mathbb{Z}$  define the *discrete interval*  $\llbracket a, b \rrbracket := \{m \in \mathbb{Z} \mid a \leq m \leq b\}$
- A *finite grid domain* is a subset  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$
- A *discrete map* is a map  $f : D \rightarrow \mathbb{R}$  over a finite grid domain  $D \subseteq \mathbb{Z}^d$
- A *d-dimensional array* is a discrete map  $f : \prod_{i=1}^d \llbracket 0, m_i \rrbracket \rightarrow \mathbb{R}$

### Definition (constant update queries):

- A discrete map  $q : D \rightarrow \mathbb{R}$  which is constant on the *update range*  $D_q = \prod_{i=1}^d \llbracket a_i, b_i \rrbracket \subseteq D$  is called *constant update query*

### Definition (constant update queries):

- A discrete map  $q : D \rightarrow \mathbb{R}$  which is constant on the *update range*  $D_q = \prod_{i=1}^d \llbracket a_i, b_i \rrbracket \subseteq D$  is called *constant update query*
- In other words,  $q = C \cdot \mathbb{1}_{D_q}$  with  $C \in \mathbb{R}$  (the *update value* of  $q$ )

### Definition (constant update queries):

- A discrete map  $q : D \rightarrow \mathbb{R}$  which is constant on the *update range*  $D_q = \prod_{i=1}^d \llbracket a_i, b_i \rrbracket \subseteq D$  is called *constant update query*
- In other words,  $q = C \cdot \mathbb{1}_{D_q}$  with  $C \in \mathbb{R}$  (the *update value* of  $q$ )
- $\text{card}(D_q)$  is the *size* of  $q$

### Definition (constant update queries):

- A discrete map  $q : D \rightarrow \mathbb{R}$  which is constant on the *update range*  $D_q = \prod_{i=1}^d \llbracket a_i, b_i \rrbracket \subseteq D$  is called *constant update query*
- In other words,  $q = C \cdot \mathbb{1}_{D_q}$  with  $C \in \mathbb{R}$  (the *update value* of  $q$ )
- $\text{card}(D_q)$  is the *size* of  $q$

Let  $f : D \rightarrow \mathbb{R}$  with  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket$  be a discrete function and  $q_1, \dots, q_L$  a series of constant update queries.

Applying the series of queries to  $f$  means forming the updated discrete function  $F : D \rightarrow \mathbb{R}$  as

$$F := f + \sum_{i=1}^L q_i.$$

## A naive algorithm

**Input:** A discrete map  $f : D \rightarrow \mathbb{R}$  with  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket$  and a series of constant update queries  $q_1, \dots, q_L$  on  $f$  over the update ranges  $D_j = \prod_{i=1}^d \llbracket a_i^{(j)}, b_i^{(j)} \rrbracket$  with update values  $C_j$  for  $1 \leq j \leq L$

**Output:** The updated function  $F = f + \sum_{i=1}^L q_i$

```
1:  $F \leftarrow$  The map  $f : D \rightarrow \mathbb{R}$ 
2: for  $j \in \llbracket 1, L \rrbracket$  do
3:   for  $k \in D_j$  do
4:      $F(k) \leftarrow F(k) + C_j$ 
5:   end for
6: end for
   return  $F$ 
```

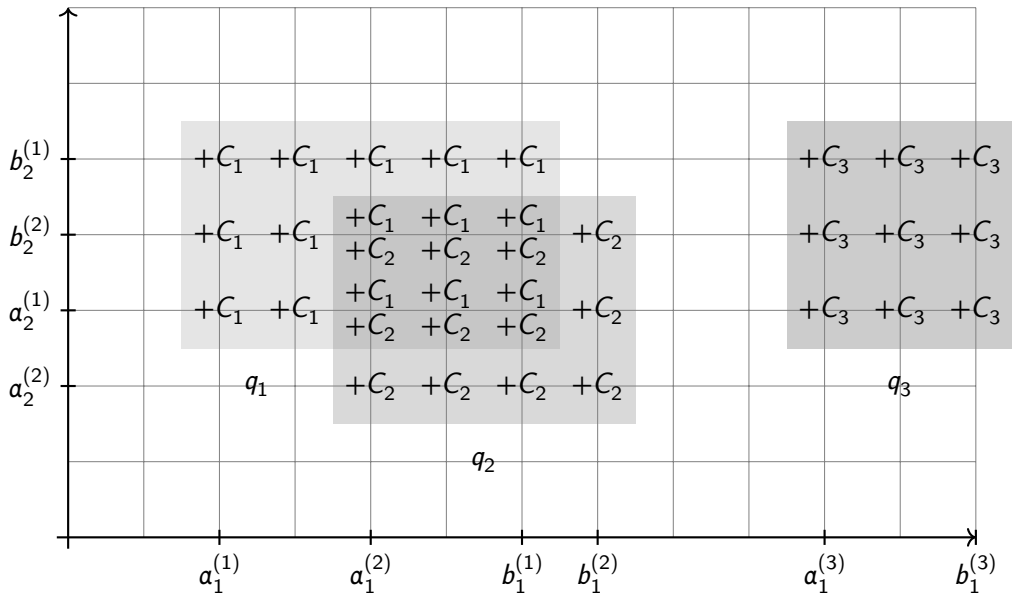
## A naive algorithm

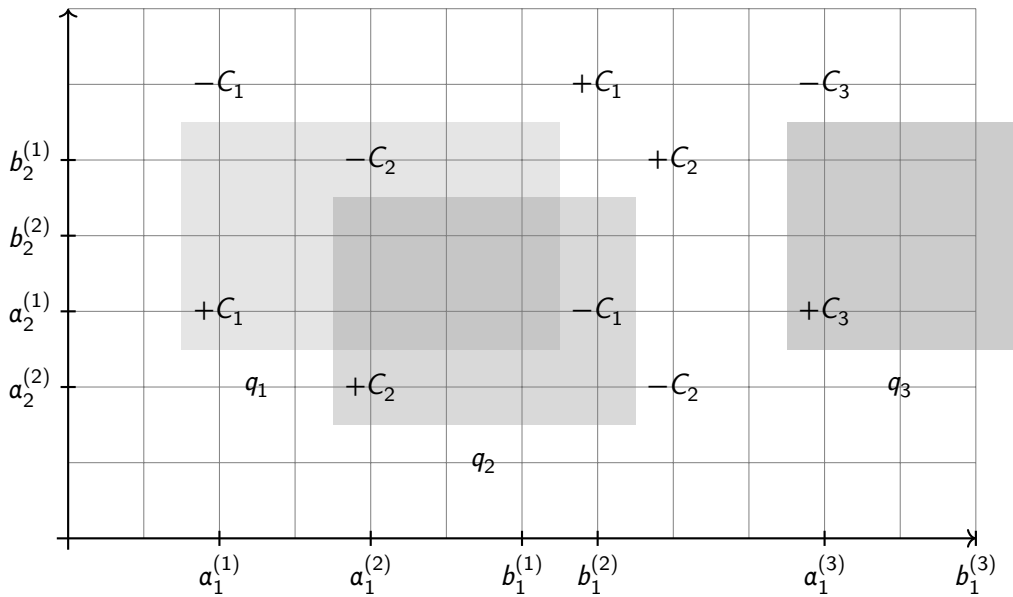
**Input:** A discrete map  $f : D \rightarrow \mathbb{R}$  with  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket$  and a series of constant update queries  $q_1, \dots, q_L$  on  $f$  over the update ranges  $D_j = \prod_{i=1}^d \llbracket a_i^{(j)}, b_i^{(j)} \rrbracket$  with update values  $C_j$  for  $1 \leq j \leq L$

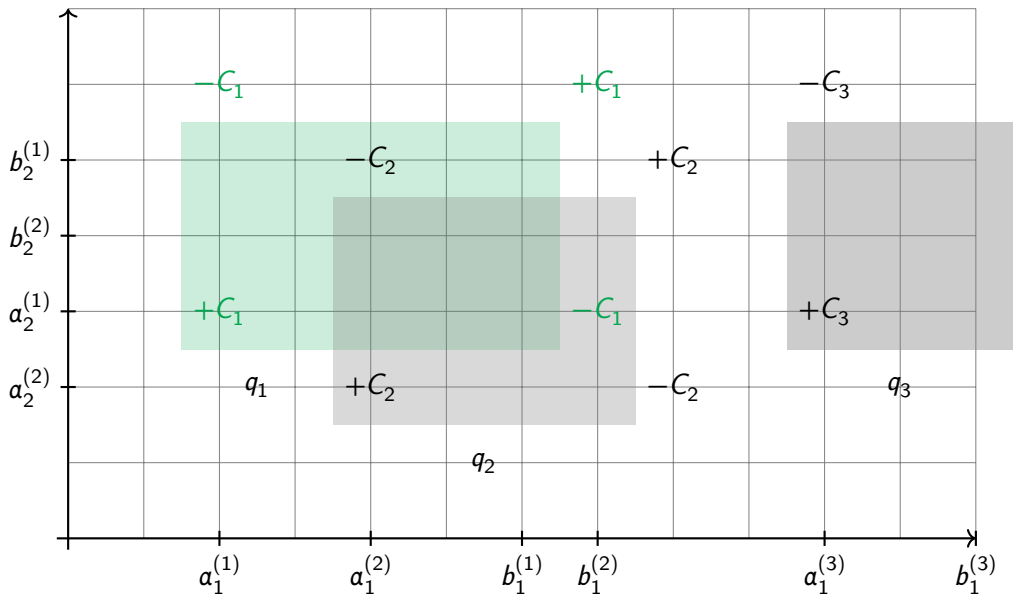
**Output:** The updated function  $F = f + \sum_{i=1}^L q_i$

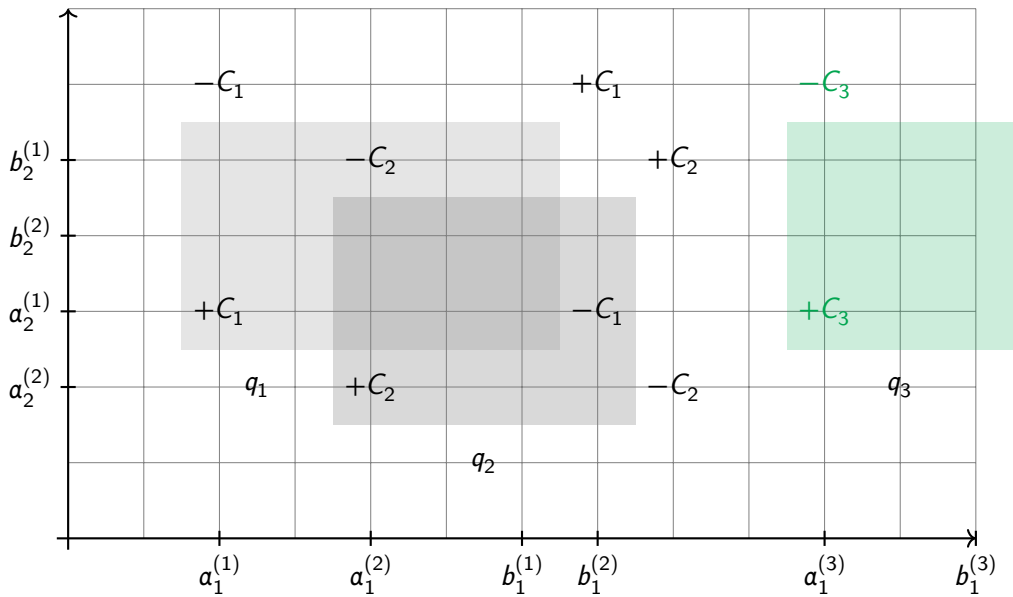
```
1:  $F \leftarrow$  The map  $f : D \rightarrow \mathbb{R}$ 
2: for  $j \in \llbracket 1, L \rrbracket$  do
3:   for  $k \in D_j$  do
4:      $F(k) \leftarrow F(k) + C_j$ 
5:   end for
6: end for
   return  $F$ 
```

**Runtime:**  $\mathcal{O}(L \cdot N)$  with  $N := \text{card}(D)$









**Definition (difference arrays):** Let

- $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain,

**Definition (difference arrays):** Let

- $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain,
- $q_1, \dots, q_L$  be a series of constant update queries on  $D$  over the update ranges  $D_j = \prod_{i=1}^d \llbracket a_i^{(j)}, b_i^{(j)} \rrbracket$  with update values  $C_j$  for  $j \in \{1, \dots, L\}$ ,

**Definition (difference arrays):** Let

- $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain,
- $q_1, \dots, q_L$  be a series of constant update queries on  $D$  over the update ranges  $D_j = \prod_{i=1}^d \llbracket a_i^{(j)}, b_i^{(j)} \rrbracket$  with update values  $C_j$  for  $j \in \{1, \dots, L\}$ ,
- $M_j := \prod_{i=1}^d \left( \{a_i^{(j)}\} \cup \{b_i^{(j)} + 1 \mid b_i^{(j)} \neq m_i\} \right)$  for  $j \in \{1, \dots, L\}$ .

**Definition (difference arrays):** Let

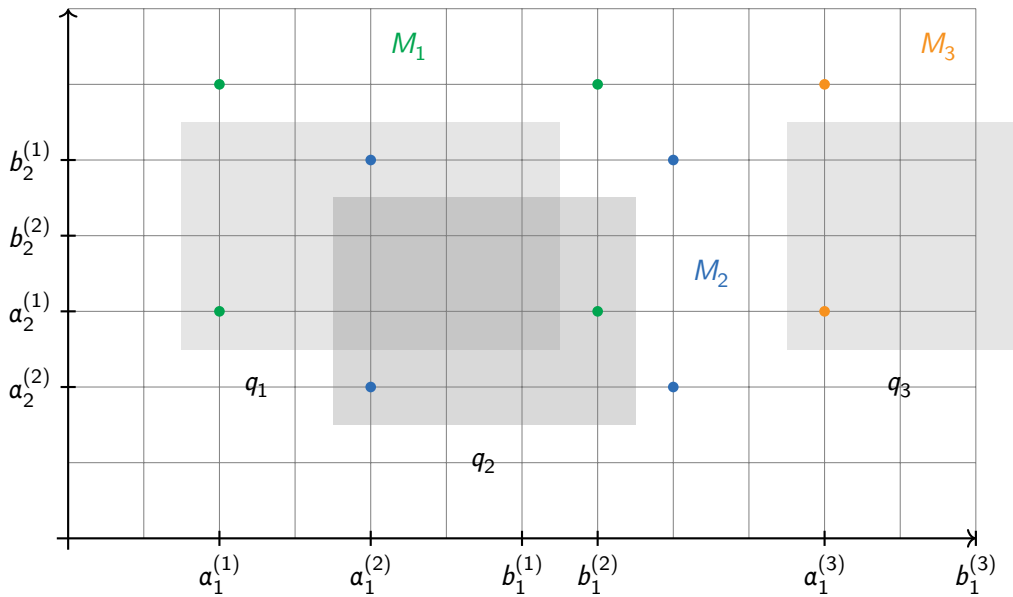
- $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain,
- $q_1, \dots, q_L$  be a series of constant update queries on  $D$  over the update ranges  $D_j = \prod_{i=1}^d \llbracket a_i^{(j)}, b_i^{(j)} \rrbracket$  with update values  $C_j$  for  $j \in \{1, \dots, L\}$ ,
- $M_j := \prod_{i=1}^d \left( \{a_i^{(j)}\} \cup \{b_i^{(j)} + 1 \mid b_i^{(j)} \neq m_i\} \right)$  for  $j \in \{1, \dots, L\}$ .

The *difference array*  $\Delta[q_1, \dots, q_L] : D \rightarrow \mathbb{R}$  is defined as

$$\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \sum_{p \in M_j} (-1)^{N_j(p)} \cdot C_j \cdot \delta_p,$$

where  $N_j(p) = \text{card}\{p_i \mid p_i = b_i^{(j)} + 1\}$  for  $j \in \{1, \dots, L\}$ .

The notation  $\delta_p$  means the Kronecker-delta.



**Procedure:** DIFFARR( $q_1, \dots, q_L$ )

**Input:** A finite grid domain  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  and a series of constant update queries  $q_1, \dots, q_L$  on  $D$  over the update ranges  $D_j = \prod_{i=1}^d \llbracket a_i^{(j)}, b_i^{(j)} \rrbracket$  with update values  $C_j$  for  $1 \leq j \leq L$

**Output:** The difference array  $\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \sum_{p \in M_j} (-1)^{N_j(p)} \cdot C_j \cdot \delta_p$

```
1:  $d \leftarrow$  Zero map  $d : D \rightarrow \mathbb{R}$ 
2: for  $j \in \{1, \dots, L\}$  do
3:    $M_j \leftarrow \prod_{i=1}^d \left( \{a_i^{(j)}\} \cup \{b_i^{(j)} + 1 \mid b_i^{(j)} \neq m_i\} \right)$ 
4:   for  $p \in M_j$  do
5:      $N(p) \leftarrow \text{card}\{p_i \mid p_i = b_i^{(j)} + 1\}$ 
6:      $d(p) \leftarrow d(p) + (-1)^{N(p)} \cdot C_j$ 
7:   end for
8: end for
9: return  $d$ 
```

**Procedure:** DIFFARR( $q_1, \dots, q_L$ )

**Input:** A finite grid domain  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  and a series of constant update queries  $q_1, \dots, q_L$  on  $D$  over the update ranges  $D_j = \prod_{i=1}^d \llbracket a_i^{(j)}, b_i^{(j)} \rrbracket$  with update values  $C_j$  for  $1 \leq j \leq L$

**Output:** The difference array  $\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \sum_{p \in M_j} (-1)^{N_j(p)} \cdot C_j \cdot \delta_p$

```
1:  $d \leftarrow$  Zero map  $d : D \rightarrow \mathbb{R}$ 
2: for  $j \in \{1, \dots, L\}$  do
3:    $M_j \leftarrow \prod_{i=1}^d \left( \{a_i^{(j)}\} \cup \{b_i^{(j)} + 1 \mid b_i^{(j)} \neq m_i\} \right)$ 
4:   for  $p \in M_j$  do
5:      $N(p) \leftarrow \text{card}\{p_i \mid p_i = b_i^{(j)} + 1\}$ 
6:      $d(p) \leftarrow d(p) + (-1)^{N(p)} \cdot C_j$ 
7:   end for
8: end for
9: return  $d$ 
```

**Runtime:** The runtime of the body of the outer for-loop is *constant*!  
 $\Rightarrow$  Total runtime:  $\mathcal{O}(N + L)$  with  $N := \text{card}(D)$

## Towards Discrete Calculus

How does this formal notion of a difference array help us in computing the updated map

$$F = f + \sum_{i=1}^L q_i \text{ more efficiently?}$$

## Towards Discrete Calculus

How does this formal notion of a difference array help us in computing the updated map

$$F = f + \sum_{i=1}^L q_i \text{ more efficiently?}$$

From the 1D/2D heuristic: Cumulatively summing over  $\Delta[q_1, \dots, q_L]$  should yield the total update  $\sum_{i=1}^L q_i$ .

## Roadmap: How does Discrete Calculus come into play?

- We will see that a difference array  $\Delta[q_1, \dots, q_L]$  is the sum of discrete derivatives of the queries  $q_1, \dots, q_L$ , that is

$$\Delta[q_1, \dots, q_L] = \sum_{i=0}^L \Delta q_i.$$

## Roadmap: How does Discrete Calculus come into play?

- We will see that a difference array  $\Delta[q_1, \dots, q_L]$  is the sum of discrete derivatives of the queries  $q_1, \dots, q_L$ , that is

$$\Delta[q_1, \dots, q_L] = \sum_{i=0}^L \Delta q_i.$$

- Cumulatively summing over  $\Delta[q_1, \dots, q_L]$  will be the discrete analogon of integration and thus will be the inverse operation to discrete differentiation, that is

$$\Sigma(\Delta[q_1, \dots, q_L]) = \sum_{i=0}^L \Sigma(\Delta q_i) = \sum_{i=0}^L q_i.$$

# Foundations of Discrete Calculus

**Definition:** The  $i$ -th backward difference  $\Delta_i$  is the linear operator assigning to every discrete map  $f : D \subseteq \mathbb{Z}^d \rightarrow \mathbb{R}$  with  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  the map  $\Delta_i f : D \rightarrow \mathbb{R}$  given by

$$\begin{aligned}\Delta_i f(k_1, \dots, k_d) &:= f(k_1, \dots, k_{i-1}, k_i, k_{i+1}, \dots, k_d) \\ &\quad - f(k_1, \dots, k_{i-1}, k_i - 1, k_{i+1}, \dots, k_d)\end{aligned}$$

for all  $(k_1, \dots, k_d) \in D$  with  $k_i > n_i$  and

$$\Delta_i f(k_1, \dots, k_d) := f(k_1, \dots, k_{i-1}, k_i, k_{i+1}, \dots, k_d)$$

for all  $(k_1, \dots, k_d) \in D$  with  $k_i = n_i$ .

**Definition (continued):** For shorthand notation, we use

$$\Delta := \Delta_1 \circ \dots \circ \Delta_d$$

to denote the *mixed backward difference*  $\Delta$ .

**Definition:** The  $i$ -th prefix sum  $\Sigma_i$  is the linear operator assigning to every discrete function  $f : D \subseteq \mathbb{Z}^d \rightarrow \mathbb{R}$  with  $D = \prod_{j=1}^d \llbracket n_j, m_j \rrbracket$  the function  $\Sigma_i f : D \rightarrow \mathbb{R}$  given by

$$\Sigma_i f(k_1, \dots, k_d) := \sum_{t=n_i}^{k_i} f(k_1, \dots, k_{i-1}, t, k_{i+1}, \dots, k_d)$$

for all  $(k_1, \dots, k_d) \in D$ . Again for shorthand notation, we use

$$\Sigma := \Sigma_1 \circ \dots \circ \Sigma_d$$

to denote the *mixed prefix sum*  $\Sigma$ .

## Properties of backward differences and prefix sums

- It is not hard to show that backward differences and prefix sums commute, that is for every  $i$  and  $j$  we have  $\Sigma_i \circ \Delta_j = \Delta_j \circ \Sigma_i$  and thus

$$\Sigma \circ \Delta = (\Sigma_1 \circ \Delta_1) \circ \cdots \circ (\Sigma_d \circ \Delta_d) = \Delta \circ \Sigma.$$

## Properties of backward differences and prefix sums

- It is not hard to show that backward differences and prefix sums commute, that is for every  $i$  and  $j$  we have  $\Sigma_i \circ \Delta_j = \Delta_j \circ \Sigma_i$  and thus

$$\Sigma \circ \Delta = (\Sigma_1 \circ \Delta_1) \circ \cdots \circ (\Sigma_d \circ \Delta_d) = \Delta \circ \Sigma.$$

- The  $i$ -th backward difference  $\Delta_i$  and the  $i$ -th prefix sum  $\Sigma_i$  are inverse to each other for all  $i$ , that is for every discrete map  $f$  we have  $\Delta_i \circ \Sigma_i(f) = f$  and thus

$$\Delta \circ \Sigma(f) = f = \Sigma \circ \Delta(f).$$

## Properties of backward differences and prefix sums

- It is not hard to show that backward differences and prefix sums commute, that is for every  $i$  and  $j$  we have  $\Sigma_i \circ \Delta_j = \Delta_j \circ \Sigma_i$  and thus

$$\Sigma \circ \Delta = (\Sigma_1 \circ \Delta_1) \circ \dots \circ (\Sigma_d \circ \Delta_d) = \Delta \circ \Sigma.$$

- The  $i$ -th backward difference  $\Delta_i$  and the  $i$ -th prefix sum  $\Sigma_i$  are inverse to each other for all  $i$ , that is for every discrete map  $f$  we have  $\Delta_i \circ \Sigma_i(f) = f$  and thus

$$\Delta \circ \Sigma(f) = f = \Sigma \circ \Delta(f).$$

- Prefix sums can be computed efficiently:

$$\begin{aligned} & \Sigma_i(f)(k_1, \dots, k_d) \\ &= f(k_1, \dots, k_{i-1}, k_i, k_{i+1}, \dots, k_d) + \sum_{t=n_i}^{k_i-1} f(k_1, \dots, k_{i-1}, t, k_{i+1}, \dots, k_d) \\ &= f(k_1, \dots, k_{i-1}, k_i, k_{i+1}, \dots, k_d) + \Sigma_i(f)(k_1, \dots, k_{i-1}, k_i - 1, k_{i+1}, \dots, k_d). \end{aligned}$$

**Procedure:** PREFIXSUM( $f, j$ )

**Input:** A discrete map  $f : D \rightarrow \mathbb{R}$  with  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  and an index  $j \in \{1, \dots, d\}$ .

**Output:** The  $j$ -th prefix sum  $\Sigma_j(f) : D \rightarrow \mathbb{R}$

```
1:  $S \leftarrow$  The discrete map  $f : D \rightarrow \mathbb{R}$ 
2: for  $(k_1, \dots, k_{j-1}, k_{j+1}, \dots, k_d) \in D_{-j} := \prod_{\substack{i=1 \\ i \neq j}}^d \llbracket n_i, m_i \rrbracket$  do
3:   for  $t \in \llbracket n_j + 1, m_j \rrbracket$  do
4:      $S(k_1, \dots, k_{j-1}, t, k_{j+1}, \dots, k_d) \leftarrow S(k_1, \dots, k_{j-1}, t - 1, k_{j+1}, \dots, k_d)$ 
5:      $\quad\quad\quad + S(k_1, \dots, k_{j-1}, t, k_{j+1}, \dots, k_d)$ 
6:   end for
7: end for
8: return  $S$ 
```

**Procedure:** PREFIXSUM( $f, j$ )

**Input:** A discrete map  $f : D \rightarrow \mathbb{R}$  with  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  and an index  $j \in \{1, \dots, d\}$ .

**Output:** The  $j$ -th prefix sum  $\Sigma_j(f) : D \rightarrow \mathbb{R}$

```
1:  $S \leftarrow$  The discrete map  $f : D \rightarrow \mathbb{R}$ 
2: for  $(k_1, \dots, k_{j-1}, k_{j+1}, \dots, k_d) \in D_{-j} := \prod_{\substack{i=1 \\ i \neq j}}^d \llbracket n_i, m_i \rrbracket$  do
3:   for  $t \in \llbracket n_j + 1, m_j \rrbracket$  do
4:      $S(k_1, \dots, k_{j-1}, t, k_{j+1}, \dots, k_d) \leftarrow S(k_1, \dots, k_{j-1}, t-1, k_{j+1}, \dots, k_d)$ 
        $+ S(k_1, \dots, k_{j-1}, t, k_{j+1}, \dots, k_d)$ 
5:   end for
6: end for
7: return  $S$ 
```

**Runtime:** The runtime of the outer for-loop lies in  $\mathcal{O}(\text{card}(D_{-j}) \cdot \text{card}(\llbracket n_j + 1, m_j \rrbracket)) = \mathcal{O}(\text{card}(D))$   
 $\implies$  Total runtime:  $\mathcal{O}(N)$  with  $N = \text{card}(D)$ .

**Procedure:** MIXEDPREFIXSUM( $f$ )

**Input:** A discrete map  $f : D \rightarrow \mathbb{R}$  with  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$

**Output:** The mixed prefix sum  $\Sigma(f) : D \rightarrow \mathbb{R}$

- 1:  $S \leftarrow$  The discrete map  $f : D \rightarrow \mathbb{R}$
- 2: **for**  $j \in \{1, \dots, d\}$  **do**
- 3:      $S \leftarrow$  PREFIXSUM( $S, j$ )
- 4: **end for**
- 5: **return**  $S$

**Procedure:** MIXEDPREFIXSUM( $f$ )

**Input:** A discrete map  $f : D \rightarrow \mathbb{R}$  with  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$

**Output:** The mixed prefix sum  $\Sigma(f) : D \rightarrow \mathbb{R}$

```
1:  $S \leftarrow$  The discrete map  $f : D \rightarrow \mathbb{R}$ 
2: for  $j \in \{1, \dots, d\}$  do
3:    $S \leftarrow$  PREFIXSUM( $S, j$ )
4: end for
5: return  $S$ 
```

**Runtime:**  $\mathcal{O}(N)$  with  $N = \text{card}(D)$

## Indicator functions and Heaviside step functions

**Definition:** Let  $p = (p_1, \dots, p_d) \in \mathbb{Z}^d$ . The *Heaviside step function*  $H_p : \mathbb{Z}^d \rightarrow \mathbb{R}$  is the indicator function given by

$$H_p(k_1, \dots, k_d) := \begin{cases} 1, & \text{if } \forall 1 \leq i \leq d : k_i \geq p_i, \\ 0, & \text{else} \end{cases}$$

for  $(k_1, \dots, k_d) \in \mathbb{Z}^d$ .

Heaviside step functions are specific indicator functions with very simple mixed backward differences.

**Proposition:** Let  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain and  $p = (p_1, \dots, p_d) \in D$ . The mixed backward difference  $\Delta H_p$  of the Heaviside step function  $H_p : D \rightarrow \mathbb{R}$  is given by

$$\Delta H_p(k_1, \dots, k_d) = \delta_p := \begin{cases} 1, & \text{if } \forall 1 \leq i \leq d : k_i = p_i, \\ 0, & \text{else.} \end{cases}$$

**Proposition:** Let  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain and  $p = (p_1, \dots, p_d) \in D$ . The mixed backward difference  $\Delta H_p$  of the Heaviside step function  $H_p : D \rightarrow \mathbb{R}$  is given by

$$\Delta H_p(k_1, \dots, k_d) = \delta_p := \begin{cases} 1, & \text{if } \forall 1 \leq i \leq d : k_i = p_i, \\ 0, & \text{else.} \end{cases}$$

**Proof.** ■ It is straight forward to see that  $H_p = \prod_{i=1}^d H_{p_i}$

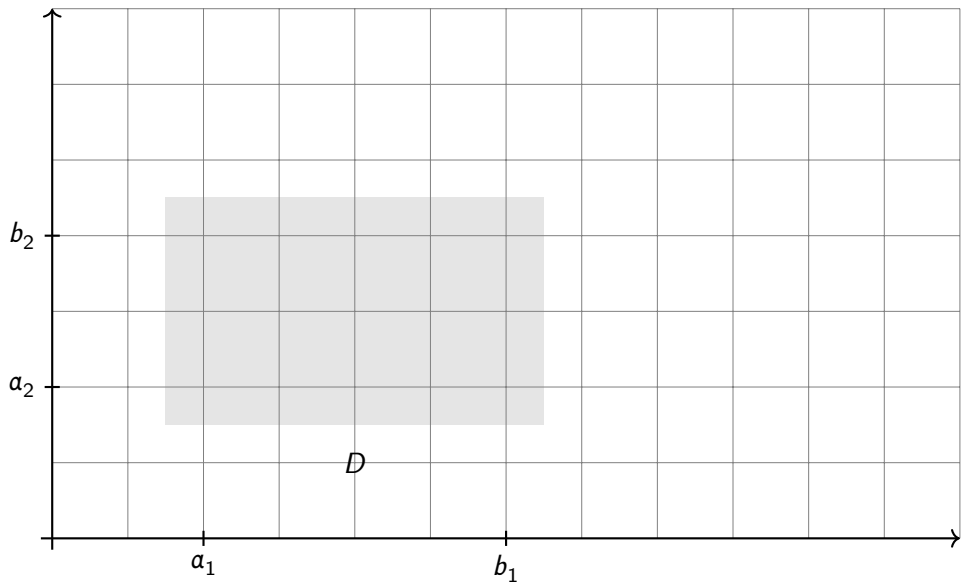
- One can show that for such products we have  $\Delta H_p = \prod_{i=1}^d \Delta H_{p_i}$
- Show  $\Delta H_{p_i} = \delta_{p_i}$  by a simple case distinction
- Finally,  $\Delta H_p = \prod_{i=1}^d \delta_{p_i} = \delta_{(p_1, \dots, p_d)}$

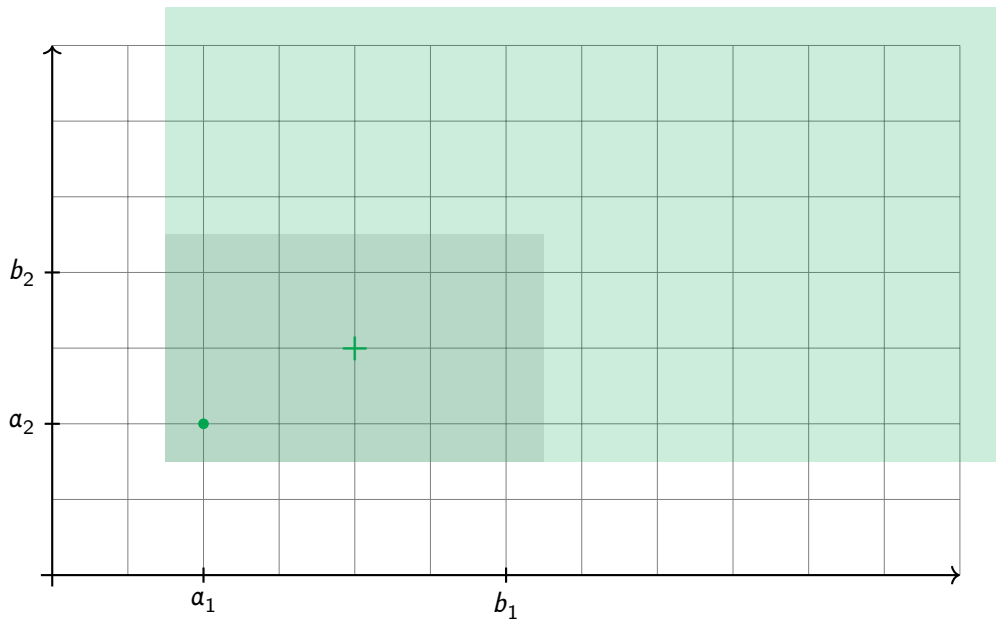
□

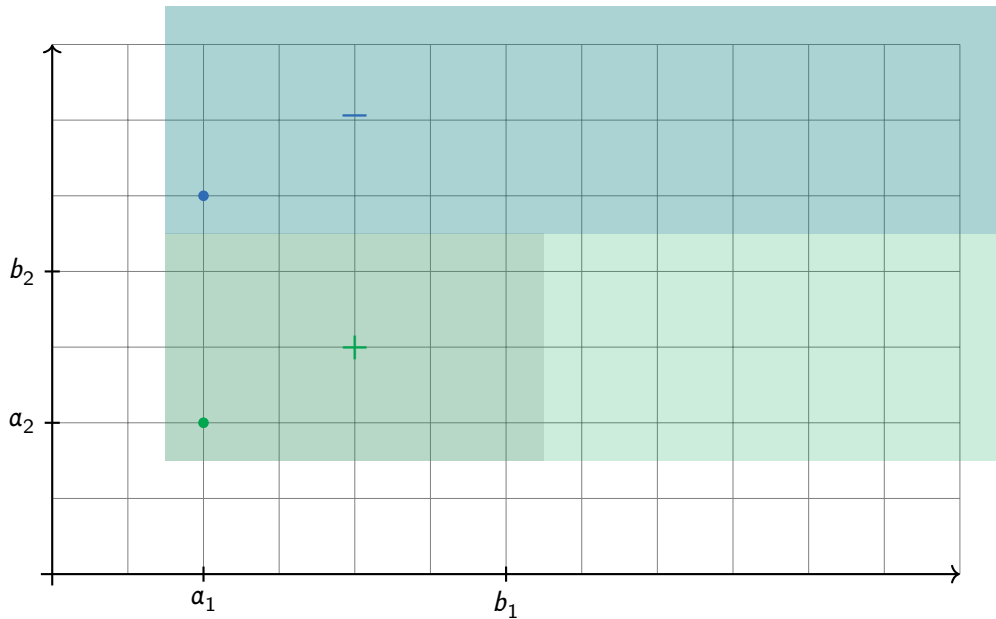
**Proposition:** Let  $D = \prod_{i=1}^d \llbracket a_i, b_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain. Then, the indicator function  $\mathbb{1}_D : \mathbb{Z}^d \rightarrow \mathbb{R}$  can be written as

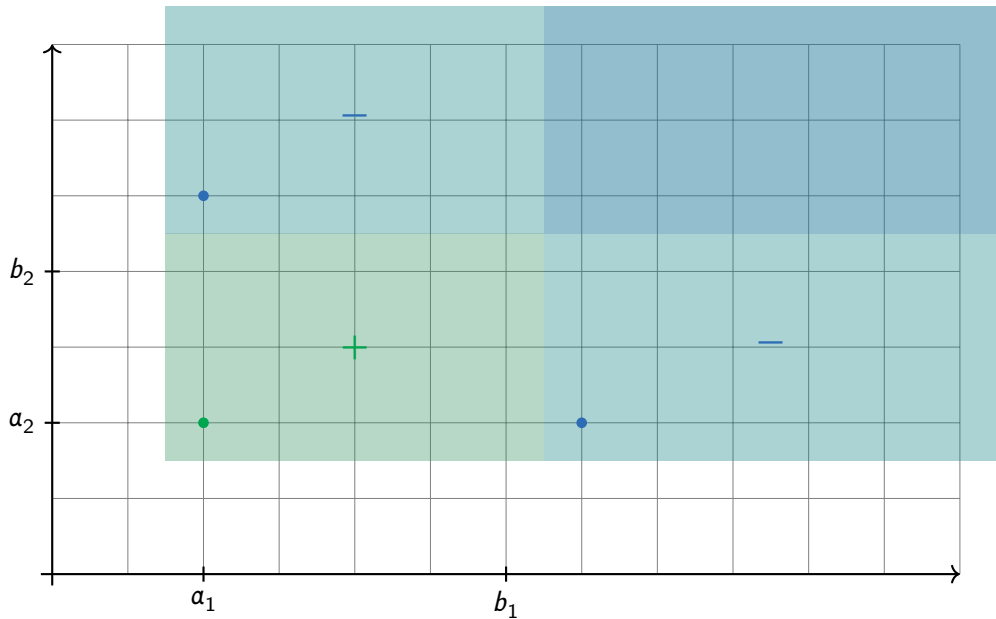
$$\mathbb{1}_D = \sum_{p \in \prod_{i=1}^d \{a_i, b_i + 1\}} (-1)^{N(p)} H_p,$$

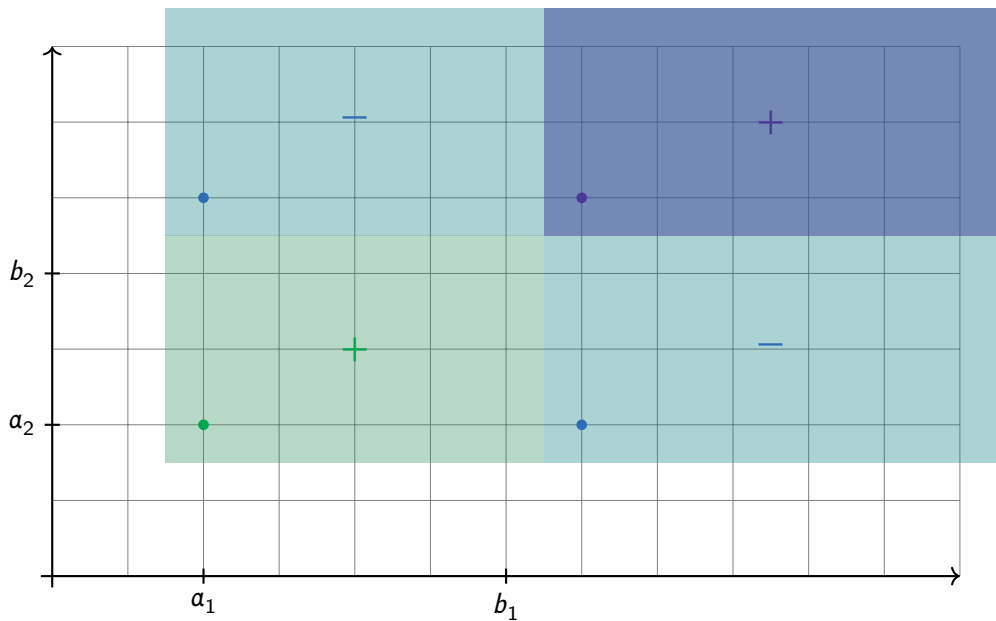
where  $N(p) = \text{card}\{p_i \mid p_i = b_i + 1\}$ .











**Proposition:** Let  $D = \prod_{i=1}^d \llbracket a_i, b_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain. Then, the indicator function  $\mathbb{1}_D : \mathbb{Z}^d \rightarrow \mathbb{R}$  can be written as

$$\mathbb{1}_D = \sum_{p \in \prod_{i=1}^d \{a_i, b_i + 1\}} (-1)^{N(p)} H_p,$$

where  $N(p) = \text{card}\{p_i \mid p_i = b_i + 1\}$ .

**Proposition:** Let  $D = \prod_{i=1}^d \llbracket a_i, b_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain. Then, the indicator function  $\mathbb{1}_D : \mathbb{Z}^d \rightarrow \mathbb{R}$  can be written as

$$\mathbb{1}_D = \sum_{p \in \prod_{i=1}^d \{a_i, b_i+1\}} (-1)^{N(p)} H_p,$$

where  $N(p) = \text{card}\{p_i \mid p_i = b_i + 1\}$ .

**Proof.** ■ It is not hard to see that  $\mathbb{1}_D(k_1, \dots, k_d) = \prod_{i=1}^d \mathbb{1}_{\llbracket a_i, b_i \rrbracket}(k_i)$

- By a simple case distinction we see that  $\mathbb{1}_{\llbracket a_i, b_i \rrbracket}(k_i) = H_{a_i}(k_i) - H_{b_i+1}(k_i)$
- Hence,  $\mathbb{1}_D(k_1, \dots, k_d) = \prod_{i=1}^d (H_{a_i}(k_i) - H_{b_i+1}(k_i))$
- Using the general distribution formula for products yields

$$\prod_{i=1}^d (H_{a_i}(k_i) - H_{b_i+1}(k_i)) = \sum_{J \subseteq I} (-1)^{\text{card}(J)} \left( \prod_{i \in I \setminus J} H_{a_i}(k_i) \right) \left( \prod_{j \in J} H_{b_j+1}(k_j) \right),$$

where  $I = \{1, \dots, d\}$ .

**Proof (continued).** ■ Then for every  $J \subseteq I$  we have

$$\left( \prod_{i \in I \setminus J} H_{a_i}(k_i) \right) \left( \prod_{j \in J} H_{b_j+1}(k_j) \right) = H_p(k_1, \dots, k_d)$$

with  $p = (p_1, \dots, p_d)$  given by

$$p_i = \begin{cases} a_i, & \text{if } i \in I \setminus J, \\ b_i + 1, & \text{if } i \in J. \end{cases}$$

- These are exactly the points in  $\prod_{i=1}^d \{a_i, b_i + 1\}$ , hence it is not hard to verify that indeed

$$\begin{aligned} \sum_{J \subseteq I} (-1)^{\text{card}(J)} \left( \prod_{i \in I \setminus J} H_{a_i}(k_i) \right) \left( \prod_{j \in J} H_{b_j+1}(k_j) \right) \\ = \sum_{p \in \prod_{i=1}^d \{a_i, b_i+1\}} (-1)^{N(p)} H_p(k_1, \dots, k_d) \end{aligned}$$

with  $N(p) = \text{card}\{p_i \mid p_i = b_i + 1\}$ .

□

**Corollary:** Let  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain and let  $q : D \rightarrow \mathbb{R}$  be a constant update query on  $D$  with update range  $D_q = \prod_{i=1}^d \llbracket a_i, b_i \rrbracket$  and update value  $C \in \mathbb{R}$ . Then,

$$q = \sum_{p \in M} (-1)^{N(p)} \cdot C \cdot H_p,$$

where  $M = \prod_{i=1}^d (\{a_i\} \cup \{b_i + 1 \mid b_i \neq m_i\})$  and  $N(p) = \text{card}\{p_i \mid p_i = b_i + 1\}$ . In particular, the mixed backward difference  $\Delta q$  of  $q$  can be expressed as

$$\Delta q = \sum_{p \in M} (-1)^{N(p)} \cdot C \cdot \delta_p.$$

**Proof.** ■ By definition  $q = C \cdot \mathbb{1}_{D_q}$

**Corollary:** Let  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain and let  $q : D \rightarrow \mathbb{R}$  be a constant update query on  $D$  with update range  $D_q = \prod_{i=1}^d \llbracket a_i, b_i \rrbracket$  and update value  $C \in \mathbb{R}$ . Then,

$$q = \sum_{p \in M} (-1)^{N(p)} \cdot C \cdot H_p,$$

where  $M = \prod_{i=1}^d (\{a_i\} \cup \{b_i + 1 \mid b_i \neq m_i\})$  and  $N(p) = \text{card}\{p_i \mid p_i = b_i + 1\}$ . In particular, the mixed backward difference  $\Delta q$  of  $q$  can be expressed as

$$\Delta q = \sum_{p \in M} (-1)^{N(p)} \cdot C \cdot \delta_p.$$

**Proof.** ■ By definition  $q = C \cdot \mathbb{1}_{D_q}$

■ By the previous proposition:

$$q = \sum_{p \in \prod_{i=1}^d \{a_i, b_i+1\}} (-1)^{N(p)} \cdot C \cdot H_p.$$

**Proof (continued).** ■ We are not viewing  $q$  as a function over  $\mathbb{Z}^d$  but over

$$D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$$

**Proof (continued).** ■ We are not viewing  $q$  as a function over  $\mathbb{Z}^d$  but over

$$D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$$

- If  $p \in \prod_{i=1}^d \{a_i, b_i + 1\}$  with  $p_j = b_j + 1$  and  $b_j = m_j$ , then  $p \notin D$

**Proof (continued).** ■ We are not viewing  $q$  as a function over  $\mathbb{Z}^d$  but over

$$D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$$

- If  $p \in \prod_{i=1}^d \{a_i, b_i + 1\}$  with  $p_j = b_j + 1$  and  $b_j = m_j$ , then  $p \notin D$
- In this case,  $H_p(k) = 0$  for all  $k \in D$

**Proof (continued).** ■ We are not viewing  $q$  as a function over  $\mathbb{Z}^d$  but over

$$D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$$

- If  $p \in \prod_{i=1}^d \{a_i, b_i + 1\}$  with  $p_j = b_j + 1$  and  $b_j = m_j$ , then  $p \notin D$
- In this case,  $H_p(k) = 0$  for all  $k \in D$
- Hence, the only points  $p \in \prod_{i=1}^d \{a_i, b_i + 1\}$  that matter are those with  $p_j \neq b_j + 1$  if  $b_j = m_j$ , that is the points in

$$M = \prod_{i=1}^d (\{a_i\} \cup \{b_i + 1 \mid b_i \neq m_i\})$$

**Proof (continued).** ■ We are not viewing  $q$  as a function over  $\mathbb{Z}^d$  but over

$$D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$$

- If  $p \in \prod_{i=1}^d \{a_i, b_i + 1\}$  with  $p_j = b_j + 1$  and  $b_j = m_j$ , then  $p \notin D$
- In this case,  $H_p(k) = 0$  for all  $k \in D$
- Hence, the only points  $p \in \prod_{i=1}^d \{a_i, b_i + 1\}$  that matter are those with  $p_j \neq b_j + 1$  if  $b_j = m_j$ , that is the points in

$$M = \prod_{i=1}^d (\{a_i\} \cup \{b_i + 1 \mid b_i \neq m_i\})$$

- Therefore,

$$q = \sum_{p \in M} (-1)^{N(p)} \cdot C \cdot H_p.$$

**Proof(continued).** ■ We are not viewing  $q$  as a function over  $\mathbb{Z}^d$  but over

$$D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$$

- If  $p \in \prod_{i=1}^d \{a_i, b_i + 1\}$  with  $p_j = b_j + 1$  and  $b_j = m_j$ , then  $p \notin D$
- In this case,  $H_p(k) = 0$  for all  $k \in D$
- Hence, the only points  $p \in \prod_{i=1}^d \{a_i, b_i + 1\}$  that matter are those with  $p_j \neq b_j + 1$  if  $b_j = m_j$ , that is the points in

$$M = \prod_{i=1}^d (\{a_i\} \cup \{b_i + 1 \mid b_i \neq m_i\})$$

- Therefore,

$$q = \sum_{p \in M} (-1)^{N(p)} \cdot C \cdot H_p.$$

- Lastly, the form of  $\Delta q$  follows directly by the linearity of the mixed backward difference  $\Delta$  and  $\Delta H_p = \delta_p$ . □

Recall the formal definition of difference arrays:

**Definition:** Let  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain and  $q_1, \dots, q_L$  a series of constant update queries on  $D$ . The *difference array*  $\Delta[q_1, \dots, q_L] : D \rightarrow \mathbb{R}$  is defined as

$$\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \sum_{p \in M_j} (-1)^{N_j(p)} \cdot C_j \cdot \delta_p,$$

where  $M_j = \prod_{i=1}^d \left( \{a_i^{(j)}\} \cup \{b_i^{(j)} + 1 \mid b_i^{(j)} \neq m_i\} \right)$  and  $N_j(p) = \text{card}\{p_i \mid p_i = b_i^{(j)} + 1\}$  for  $1 \leq j \leq L$ .

Recall the formal definition of difference arrays:

**Definition:** Let  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket \subseteq \mathbb{Z}^d$  be a finite grid domain and  $q_1, \dots, q_L$  a series of constant update queries on  $D$ . The *difference array*  $\Delta[q_1, \dots, q_L] : D \rightarrow \mathbb{R}$  is defined as

$$\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \sum_{p \in M_j} (-1)^{N_j(p)} \cdot C_j \cdot \delta_p,$$

where  $M_j = \prod_{i=1}^d \left( \{a_i^{(j)}\} \cup \{b_i^{(j)} + 1 \mid b_i^{(j)} \neq m_i\} \right)$  and  $N_j(p) = \text{card}\{p_i \mid p_i = b_i^{(j)} + 1\}$  for  $1 \leq j \leq L$ .

- With the corollary on the last slide we see that

$$\Delta q_j = \sum_{p \in M_j} (-1)^{N_j(p)} \cdot C_j \cdot \delta_p$$

for all  $j \in \{1, \dots, L\}$ .

- Hence, we indeed have

$$\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \Delta q_j.$$

- Hence, we indeed have

$$\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \Delta q_j.$$

- The prefix sum and the mixed backwards difference are inverse to each other, thus

$$\Sigma(\Delta[q_1, \dots, q_L]) = \sum_{j=1}^L \Sigma(\Delta q_j) = \sum_{j=1}^L q_j.$$

- Hence, we indeed have

$$\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \Delta q_j.$$

- The prefix sum and the mixed backwards difference are inverse to each other, thus

$$\Sigma(\Delta[q_1, \dots, q_L]) = \sum_{j=1}^L \Sigma(\Delta q_j) = \sum_{j=1}^L q_j.$$

- We have seen that  $\Delta[q_1, \dots, q_L]$  can be computed in  $\mathcal{O}(L + N)$ .

- Hence, we indeed have

$$\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \Delta q_j.$$

- The prefix sum and the mixed backwards difference are inverse to each other, thus

$$\Sigma(\Delta[q_1, \dots, q_L]) = \sum_{j=1}^L \Sigma(\Delta q_j) = \sum_{j=1}^L q_j.$$

- We have seen that  $\Delta[q_1, \dots, q_L]$  can be computed in  $\mathcal{O}(L + N)$ .
- We have seen that the prefix sum  $\Sigma(\Delta[q_1, \dots, q_L])$  can be computed in  $\mathcal{O}(N)$ .

- Hence, we indeed have

$$\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \Delta q_j.$$

- The prefix sum and the mixed backwards difference are inverse to each other, thus

$$\Sigma(\Delta[q_1, \dots, q_L]) = \sum_{j=1}^L \Sigma(\Delta q_j) = \sum_{j=1}^L q_j.$$

- We have seen that  $\Delta[q_1, \dots, q_L]$  can be computed in  $\mathcal{O}(L + N)$ .
- We have seen that the prefix sum  $\Sigma(\Delta[q_1, \dots, q_L])$  can be computed in  $\mathcal{O}(N)$ .
- The addition  $f + \Sigma(\Delta[q_1, \dots, q_L])$  has time complexity  $\mathcal{O}(N)$ .

- Hence, we indeed have

$$\Delta[q_1, \dots, q_L] = \sum_{j=1}^L \Delta q_j.$$

- The prefix sum and the mixed backwards difference are inverse to each other, thus

$$\Sigma(\Delta[q_1, \dots, q_L]) = \sum_{j=1}^L \Sigma(\Delta q_j) = \sum_{j=1}^L q_j.$$

- We have seen that  $\Delta[q_1, \dots, q_L]$  can be computed in  $\mathcal{O}(L + N)$ .
- We have seen that the prefix sum  $\Sigma(\Delta[q_1, \dots, q_L])$  can be computed in  $\mathcal{O}(N)$ .
- The addition  $f + \Sigma(\Delta[q_1, \dots, q_L])$  has time complexity  $\mathcal{O}(N)$ .
- In total: Computing  $f + \Sigma(\Delta[q_1, \dots, q_L])$  can be done in  $\mathcal{O}(L + N)$ .

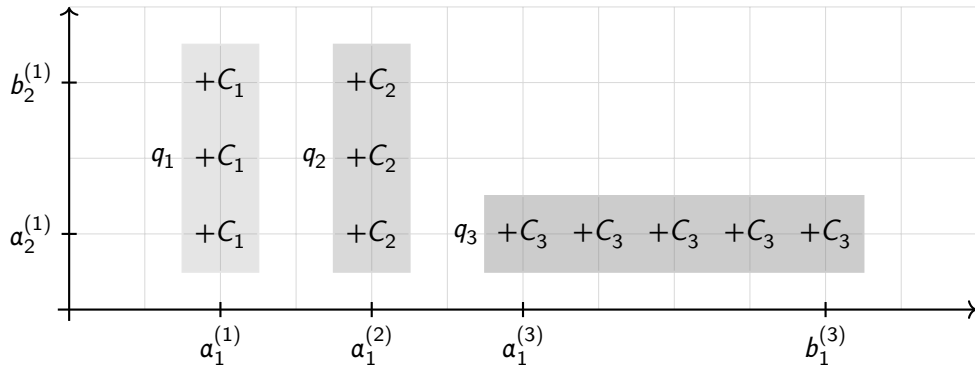
**Input:** A discrete map  $f : D \rightarrow \mathbb{R}$  with  $D = \prod_{i=1}^d \llbracket n_i, m_i \rrbracket$  and a series of constant update queries  $q_1, \dots, q_L$  on  $D$

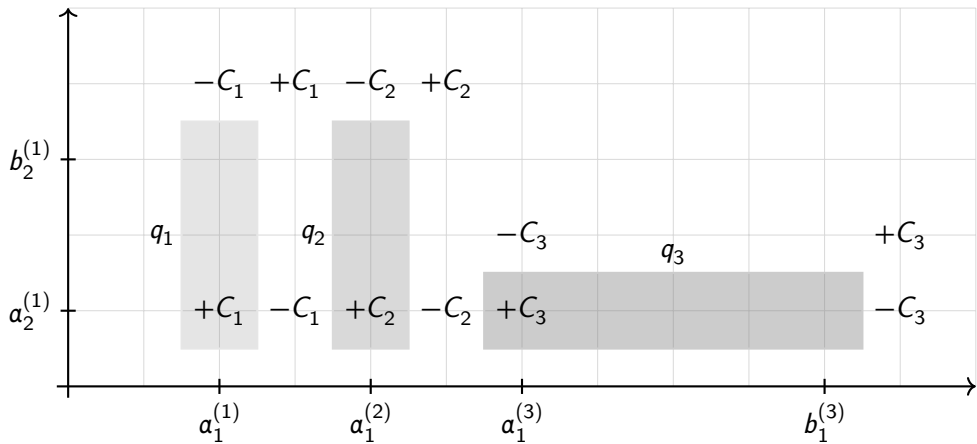
**Output:** The updated map  $F = f + \sum_{i=1}^L q_i$

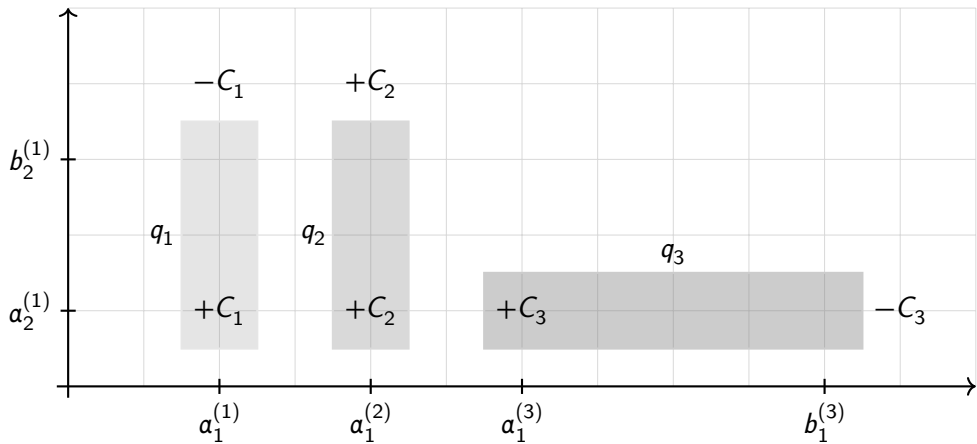
- 1:  $d \leftarrow \text{DIFFARR}(q_1, \dots, q_L)$
- 2:  $\Sigma d \leftarrow \text{MIXEDPREFIXSUM}(d)$
- 3:  $F \leftarrow f + \Sigma d$
- 4: **return**  $F$

**Runtime:**  $\mathcal{O}(L+N+N+N) = \mathcal{O}(L+N)$   
with  $N := \text{card}(D)$

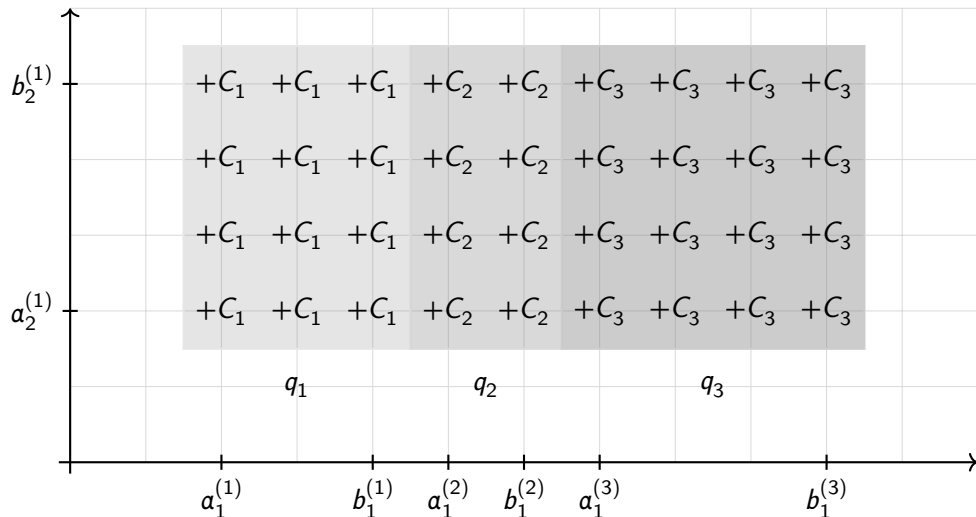
## Flat update queries

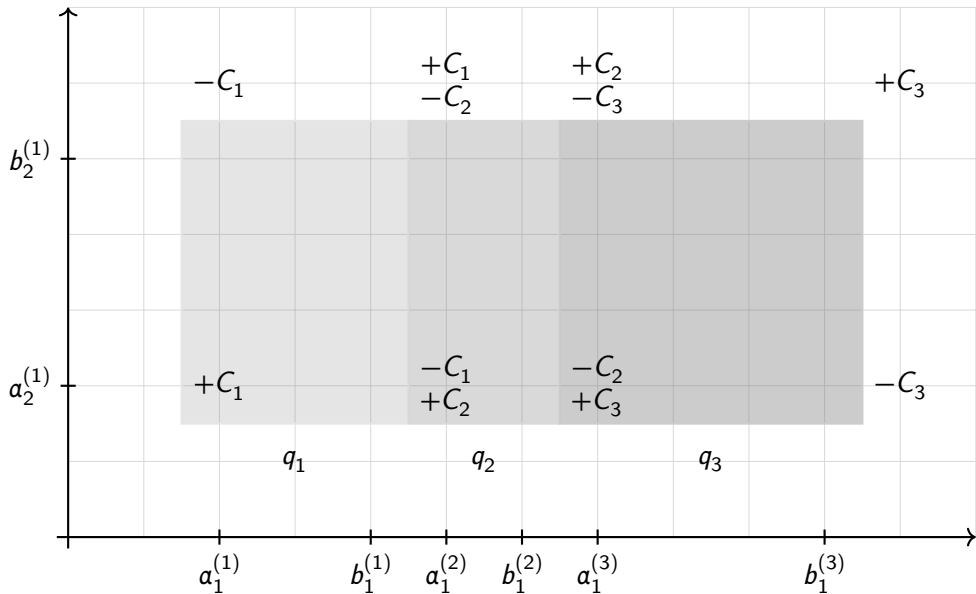




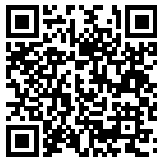


## Consecutive update queries





# Any questions?



(<https://github.com/mazmap/multidimensional-difference-arrays>)

↑ Slides ↑